



Polyline Morphing for Animated Schematic Maps

Axel Forsch^{1,2} · Ruben Kemna^{1,3} · Elmar Langetepe^{1,3} · Jan-Henrik Haunert^{1,2}

Accepted: 12 September 2024 / Published online: 8 October 2024
© The Author(s) 2024

Abstract

Thematic maps allow for the visual analysis of spatial data. When comparing two map states, preserving the mental map of a user facilitates the comparison. One way to achieve this is to use animated transitions between the states. This work presents an algorithm for computing such animations, called morphs, between schematized map objects, a technique particularly pertinent in urban mobility scenarios where schematic maps improve map legibility. In schematic maps, abstraction is used to reduce the visual complexity while still conveying information on a selected phenomenon. Our method ensures that the morph has four favorable properties: (1) it is self-intersection-free, (2) it maintains the schematization of the input features, (3) it is self-contained, and (4) every segment moves at its own constant velocity. We present an efficient algorithm to compute vertex traces and the timing of the morph. We evaluate our approach on isochrones visualizing travel times and on different layouts of schematic transit networks. The results show that the additional constraints we induce on the morphing only have a minor influence on the optimization target while they reduce the complexity of the animation.

Keywords Morphing · Schematic maps · Animation · Optimization · Isochrones

Introduction

In dynamic maps, spatio-temporal changes and spatial processes are communicated using animated transitions between different map states (Dransch 2001; Battersby and Goldsberry 2010; Harrower 2001). The transitions are often referred to as *morphs*, and they are used to support the user in detecting important visual changes that occur during the scene transition.

While the animation can support the detection of changes in the maps, they also add complexity to the visualization, which in extreme cases may cause a cognitive overload for the user, thus limiting the information the user processes (Morrison 2000). Thus, animated maps should be generalized to

the most important information to simplify the visualization (Harrower 2003).

Schematic mapping is a cartographic concept that deals with reducing the visual complexity of maps. In schematized maps, a simple and quick-to-understand visualization over a geometrically accurate representation of the features is favored. In this work, we present a method for combining dynamic and schematic maps by generating a morph between two schematized input polylines. Our approach is designed specifically for polylines in which the orientation of each segment is limited to a prescribed set of orientations. One example of such a schematization is octilinear polylines, where all segments are horizontal, vertical, or diagonal. For instance, this type of schematization is widely applied to maps of metro networks (Wu et al. 2020).

The schematization should be preserved during the morph to reduce the complexity of the animation. Phrased differently, the orientation of the segments of the *intermediate polylines* that appear in each frame of the animation should be restricted to the same set of orientations as the input polylines. In the case of octilinear input polylines, all intermediate polylines are also octilinear. The morph is schematized by restricting polyline segments to move parallelly without changing orientation. Moreover, we ensure that every segment moves at its own, constant velocity during the morph, further reducing the visual complexity. An example is shown in Fig. 1.

Animated versions for all figures in this article can be found online at <https://www.geoinfo.uni-bonn.de/morphing>.

✉ Axel Forsch
forsch@igg.uni-bonn.de

¹ University of Bonn, 53115 Bonn, Germany

² Institute of Geodesy and Geoinformation, Meckenheimer Allee 172, Bonn, Germany

³ Institute of Computer Science, Endenicher Allee 19a, Bonn, Germany

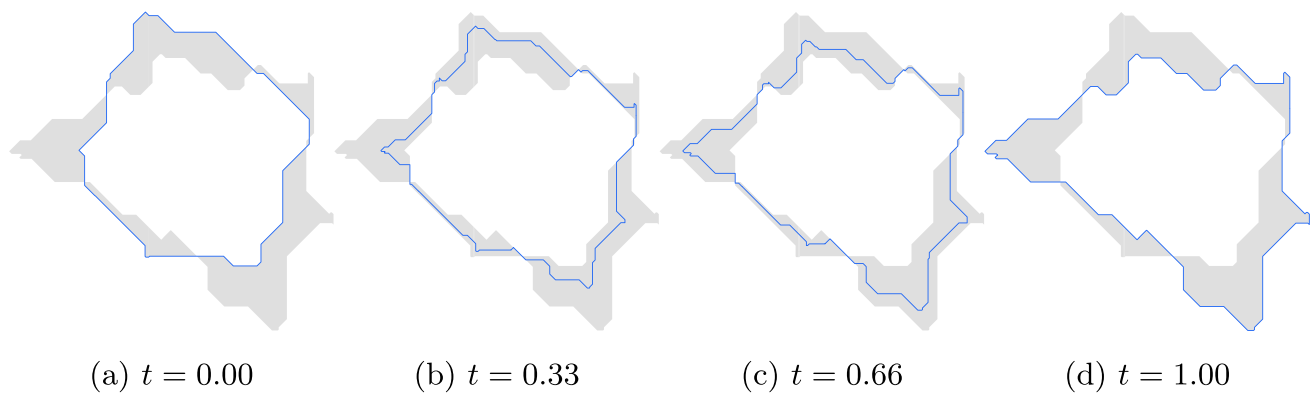


Fig. 1 Our result for a morph between two schematic isolines (Animated versions for all figures in this article can be found online at <https://www.geoinfo.uni-bonn.de/morphing>). All polylines are schematic, self-intersection-free, and contained in the morph region (gray)

This article proposes an efficient algorithm for morphing schematic input polylines. Specifically, our algorithm produces meaningful morphs for real-world applications by requiring that the morph stays between the input polylines. We show that our algorithm can easily be extended to morph between polygons and between inputs of varying schematization types, i.e., different sets of orientations. The usefulness of the morphs is evaluated for two types of thematic maps, both of which are related to urban mobility.

The first one is isochrone maps, where travel times are displayed by *isochrones* that enclose the region reachable from a given starting position within a given travel time. We use our morphing algorithm on isochrones to visualize different cycling routing profiles. To this end, we broaden the definition of isochrones to lines containing all points that can be reached with equal *effort* from a given origin. Figure 1 displays an example morph generated for two routing profiles that reach differently far in varying directions.

The second example is transit network maps, which are well-known for their schematic nature. Specifically, we look at the Focus+Context method developed by Wang and Chi (2011). In their work, the authors generate different transit network layouts based on the selected route. We use our

morphing algorithm to animate the transition between these layouts. Figure 2 displays an example morph generated for two transit network layouts for the city of Stockholm.

To summarize our contribution, we present an efficient algorithm that, for two schematic polylines, yields a morph such that it has four favorable properties: (1) it maintains the schematization of the input features, (2) it is self-intersection-free, (3) it is self-contained, and (4) it has constant velocity. To the best of our knowledge, this is the first morphing algorithm that is designed for schematic inputs under challenging constraints such as timing, simplicity, and topology. Alongside this article, the implementation is published as open-source code. It can be accessed at <https://gitlab.igg.uni-bonn.de/rukemn/traveltimemaps>. Moreover, animated versions for all figures in this article can be found online at <https://www.geoinfo.uni-bonn.de/morphing>.

Related Work

The concept of morphing was first introduced in the computer graphics community to warp one image into another. In

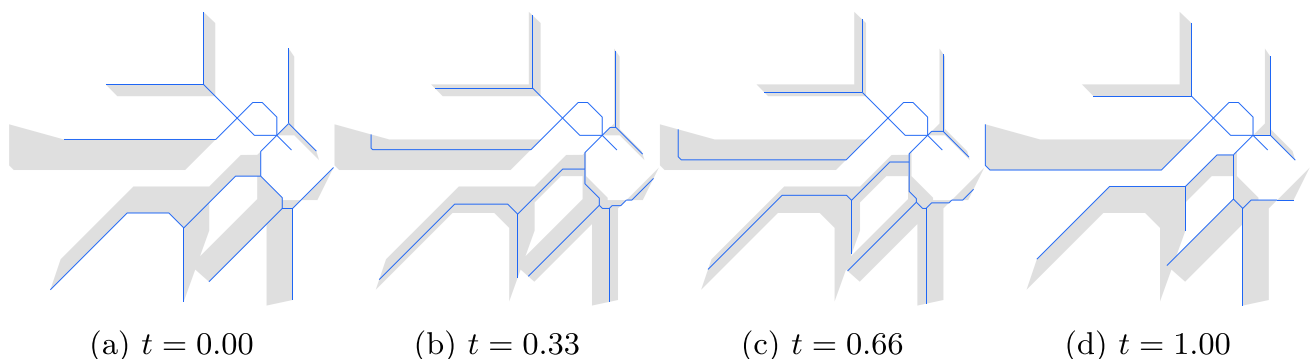


Fig. 2 Our result for a morph between two layouts of a schematic transit map. Input layouts are adapted from the Focus+Context Metro Map approach (Wang and Chi 2011)

this context, two main methodologies emerged: pixel blending and shape warping. In pixel blending, the pixel values of the first image are gradually shifted toward the second image (Rahman et al. 2007; Kekre et al. 2011). As intermediate states generated by this method show gray values or colors not in the original image, this method is unsuitable for thematic maps with discrete color scales. Nonetheless, Lin and Gong (2018) present an approach similar to pixel blending based on cellular automata suitable for thematic maps.

Shape Warping In thematic maps, two-dimensional shape warping, where one shape is transformed smoothly into another, is relevant. For the morphing of polygons, Surazhsky and Gotsman (Surazhsky and Gotsman 2001a; Gotsman and Surazhsky 2001; Surazhsky and Gotsman 2001b) use compatible triangulations to guarantee that intermediate polygons are self-intersection-free. However, this method tends to produce long vertex traces. As an alternative, a morphing approach based on *Hausdorff middles* has been presented (van Kreveld et al. 2022). The Hausdorff middle of two shapes is a set that minimizes the maximum Hausdorff distance between them. The authors generate a morph by recursively computing the Hausdorff middle of the two input shapes. A drawback of this approach is that the Hausdorff middle of two polygons is not necessarily a polygon itself. Li et al. (2018) compute a morphing in the frequency space by first applying a Fourier transform on the input data and then computing an intermediate function in the Fourier space.

Another approach to shape warping is by modeling it as a two-step process. First, correspondences between the points of the shapes are established. Secondly, the movement between corresponding points is specified. By using appropriate correspondences, the change during the morph can be optimized. Early works that adopt this method use linear point paths (Sederberg and Greenwood 1992). A problem with this approach is that it can lead to self-intersections in the intermediate shapes. For the morph of two non-intersecting, simple polylines F and G with n vertices, the method of Efrat et al. (2001) guarantees that all intermediate polylines are self-intersection-free by using shortest paths that avoid the input polylines for the point movement. In several publications, improvements of this approach are proposed, resulting in a running time of $\mathcal{O}(n^2)$ for an optimal solution (Bespamyatnikh 2002) and a running time of $\mathcal{O}(n)$ for a solution with an approximation factor of two (Bespamyatnikh 2003; Bereg 2005).

Polyline Correspondence Much attention is given to the correspondence between the two polylines F and G . Most commonly, the correspondence is modeled by two continuous, monotone parameterizations $\Phi: [0, 1] \rightarrow F$ and $\Gamma: [0, 1] \rightarrow G$. For each $u \in [0, 1]$, the point $\Phi(u)$ on the source polyline corresponds to point $\Gamma(u)$ on the target poly-

line. An important feature for finding the correspondence is the similarity of polylines. Similar sections on the two polylines should also correspond in the morph. One of the earliest similarity measures between curves is the Fréchet distance (Alt and Godau 1995). Efrat et al. (2002) introduce two Fréchet-like similarity measures designed explicitly for polyline morphing, the geodesic width, i.e., the length of the maximum point movement, and the link width, i.e., the maximum number of links of the geodesic point movements. Buchin et al. (2019) introduce a method to compute locally correct Fréchet matchings, where matchings for every two subpolylines are valid Fréchet matchings again. Liu et al. (2004) extract perceptual feature points of the input shapes and perform the correspondence based on properties of the local neighborhood of each feature point.

Morphing and Cognition The utility of animation in human perception has been extensively explored within cognitive science. Archambault and Purchase (2016) examined how animations enhance graph drawings, revealing that animation significantly improves task performance, particularly in scenarios with low stability. This finding supports earlier research by Tversky et al. (2002). Additionally, Sedig et al. (2005) compared different design interfaces for transitional processes and found that animations are the most effective to aid users in constructing cognitive maps.

Another research area focuses on how intermediate shapes during a morph are perceived (Panis et al. 2008). Several studies have identified discrimination thresholds, which are points within the morph space where differences in shape become noticeable (Destler et al. 2019; Hartendorp et al. 2010). Godfrey and Mackaness (2017) applied this concept to geographical information, investigating the maximum extent of map distortions that still permit effective use of the maps.

Morphing for Map Comparison The comparison of maps is an important topic in cartography (Visser and de Nijs 2006; Lobo et al. 2015) that is frequently used to analyze urban mobility (Polisciuc et al. 2013; Dewulf et al. 2015). Gortana et al. (2014) present a web application for visually exploring urban mobility patterns based on isochrones. In the outlook, the authors highlight the need for smooth transitions to improve the exploration. An important concept in this context is the cognitive or mental map, which reflects the users' perception of an environment (Kitchin 1994). Preserving the mental map has been shown to improve the user's orientation (Archambault and Purchase 2013). Previous studies show that animations can help to build and preserve the mental map (Bederson and Boltman 1999; Diehl et al. 2001).

Morphing in Map Generalization In map generalization, morphing is used to create a smooth transition between different zoom levels (van Kreveld 2001; Cecconi et al.

2002; Pantazis et al. 2009; Kada et al. 2015; Li et al. 2017a; Gao et al. 2020). Nöllenburg et al. (2008) present a method to detect characteristic points of polylines, which are then used as input for the correspondence problem. They present a dynamic programming approach for computing the correspondence and a heuristic method to avoid self-intersections in the intermediate states. Their approach is similar to Dynamic Time Warping, which has also been applied to the continuous generalization of linear features (Li and Mao 2023). Similarly, Deng and Peng (2015) match polylines based on the similarity of the bend structures to generate morphs between features of different scales. Sester and Brenner (2005) consider the generalization of building footprints and highlight the need to preserve the object regularities. Based on a given footprint, the authors generate simplified variants of the footprint while simultaneously generating a morph between the shapes. Their rule-based approach guarantees a rectilinear simplification of building footprints. Similarly, Li et al. (2017b) present an angle-preserving morphing approach for building footprints by establishing the correspondence in the turning-angle function feature space. Similar to the previous approach, their method can only simplify the input feature. van Oosterom and Meijers (2014) introduce the *space-scale cube* (SSC), a data structure for the continuous generalization of geographic features. More recently, Peng et al. (2023) use a SSC to generate a smooth zoom animation based on merging area objects.

Schematization In cartography, *schematization* highlights specific aspects of a map while deemphasizing others (Klippel et al. 2005; Roberts 2023). Schematization aims to reduce the complexity of the map (Meulemans 2014; Ti et al. 2023) and is often used in combination with geographic base maps (Böttger et al. 2008). This article adopts the concept of *C*-oriented schematization. The edges of a *C*-oriented shape are restricted to a set *C* of edge orientations. Two popular variants of *C*-oriented schematization are rectilinear shapes, where all edges are horizontal or vertical, and octilinear shapes, where diagonal edges are allowed next to the rectilinear directions. Dellling et al. (2010) apply *C*-oriented schematization to create sketches of routes. The goal is to create a schematized variant of the input path that retains key properties, such as being intersection-free and preserving the orthogonal order, i.e., the relative location of consecutive locations. Similarly, Speckmann and Verbeek (2018) present algorithms for finding *C*-oriented schematizations for input paths that preserve the relationship of the path to given obstacles. Early works for the schematization of polygonal shapes included *O*-hulls, a schematized convex hull of the polygon (Fink and Wood 2004). Cicerone and Cermignani (2012) present a heuristic for rectilinear and octilinear schematization of polygons. Instead of considering single polygons, some methods deal with the schematization of polygonal subdivisions that pre-

serve the topology of neighboring polygons (Meulemans et al. 2010; Buchin et al. 2016). Bonerath et al. (2019) present a data structure that supports time-window queries on large sets of points with time stamps. The result of a query is a schematic polygonal representation of the set of all points within the queried time window. Thus, *C*-oriented schematization has been applied in all three vector data domains: point, line, and area.

Transit Maps A well-known application domain of schematization is that of transit map layouts (Jenny 2006; Wu et al. 2020), which, in their schematized form, are often referred to as *metro maps*. Much research was done in this context on the automatic schematization of networks (Li and Dong 2010; Nöllenburg and Wolff 2011; van Dijk and Lutz 2018; Bast et al. 2020; Brosi and Bast 2024). Li (2015) formulates four general principles for the schematization of networks: preservation of the topological relationship, preservation of the main structure of a line network, relativity in position, and relativity in length. Based on these principles, Lan et al. (2019) present a mixed-integer approach for the construction of octilinear metro maps. Taking it a step further, Nöllenburg and Terziadis (2024) present a method for the automatic generation of *C*-oriented metro maps, which dynamically selects the set *C* from the input data.

Wayfinding Wayfinding is another domain where schematization has frequently been applied (Casakin 2004; Meilinger et al. 2007; Speckmann and Verbeek 2018; Galvão et al. 2021). Galvão et al. (2023) present an approach for schematizing a given route and the surrounding transportation network simultaneously. Wang and Chi (2011) present a focus+context approach that allows highlighting a specific route in a metro network by deforming the network such that the selected route is enlarged. One application domain for our morphing scheme could be the transition from the original metro map to the metro map with a highlighted route.

Although previous publications have given considerable attention to polyline morphing and schematization, none of the existing morphing algorithms maintain the schematic properties during animation.

Base Case: Morphing Simple Polylines

This section introduces the notation used throughout the article and establishes the algorithmic problem we are solving. Then, an algorithm for solving the problem for the base case for morphing two non-intersecting polylines is presented. In the “General Case” section, we introduce simple extensions to the algorithm to make it applicable to closed rings, i.e., polyline outlines, that may also intersect.

Notation A polyline *L* is represented as a sequence of points $\langle l_0, l_1, \dots, l_k \rangle$, which are the vertices of *L*. A straight-line

segment connecting two consecutive vertices in L is called a *segment* of L . We require that L is C -oriented, meaning that the orientation of every segment of L stems from a predefined set C of orientations. If L has no self-intersections, we call it simple. We denote the subpolyline of L that starts at l_i and ends at l_j with $i \leq j \leq k$ as L_i^j . Further, the length of L , i.e., the sum of its segments' lengths, is denoted as $\ell(L)$. We call the shortest path from a point s to a point t avoiding a given set of obstacles $\mathcal{O}$ the *geodesic path* $\mathcal{G}_{\mathcal{O}}(s, t)$.

Input As input, we take two simple schematic polylines $F = \langle f_1, \dots, f_n \rangle$ and $G = \langle g_1, \dots, g_m \rangle$. We call F the *source* and G the *target*. In a basic setting, we assume (i) that F and G do not cross each other and (ii) that $S = \mathcal{G}_{\{F, G\}}(f_1, g_1)$ and $T = \mathcal{G}_{\{F, G\}}(f_n, g_m)$, i.e., the geodesic paths connecting the input polylines' endpoints while not crossing them, do not intersect. As a result, the polylines F , T , G , and S can be concatenated to form the outline of a simply connected region (Tang 2007). We call this region the *morph region* and denote it as R . Figure 3 illustrates the definition of the morph region and provides examples of its construction for different configurations that may arise that fulfill assumptions (i) and (ii). In the “General Case” section, we explain simple preprocessing steps, allowing us to deal with inputs where assumptions (i) and (ii) do not hold.

Problem We aim to find a function $M(t)$ that returns for every timestamp $t \in [0, 1]$ a simple schematic polyline such that $M(0) = F$ and $M(1) = G$. We impose the natural requirement that M is continuous. Further, for every $t \in [0, 1]$, the polyline $M(t)$ must fulfill the following properties:

- (1) **SCHEMATIC**: the segments of $M(t)$ are restricted to the set C of directions that the segments of F and G are restricted to.
- (2) **SELF- INTERSECTION- FREE**: $M(t)$ is self-intersection free.
- (3) **SELF- CONTAINED**: $M(t)$ is within the morph region R .
- (4) **CONSTANT VELOCITY**: the velocity of a moving segment in $M(t)$ is constant.

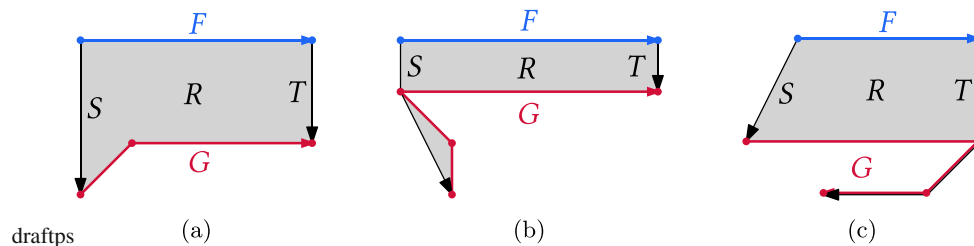


Fig. 3 The morph region R is defined by the two input polylines F and G along with the geodesic paths S and T connecting their endpoints. In the standard case (a), the morph region is a simple polygon, but depend-

We call M a *morph* between its source F and target G . The requirements for M reduce the visual complexity of the morph.

Similar to previous work in polyline morphing, we split the morphing into two problems. At first, we find a correspondence between every point on F to a point on G . We call this the *correspondence problem*. The second problem is to specify the timing of the movement of points on F to their corresponding points on G . We call this the *timing problem*. The following gives an overview of our solution to these two problems.

Correspondence

We consider the correspondence as a discrete problem by finding the correspondences of vertices between the two polylines. It is sufficient to model the correspondences for all vertices (corners) of F and G . We reason this because we are dealing with schematized input polylines. Schematization aims to reduce the complexity of the non-schematized feature while preserving its characteristic properties. Therefore, we can assume that the input polylines' vertices capture the features' characteristics.

We call the path a point $f \in F$ takes during the animation to get to point $g \in G$ the *trace* of f and g . The simplest way of defining the traces is by using straight-line movements with uniform motion, i.e., the points have constant speed along their trace. Although this approach is established, it can lead to unwanted self-intersections in M . This problem can be avoided by using geodesic paths inside the morph region R as traces (Efrat et al. 2001). Therefore, we use such geodesic paths as traces, making them uniquely defined by the vertex correspondences.

Vertex-Based Correspondence

We model the vertex-based correspondence of the two polylines with a sequence $\Pi = (\pi_1, \dots, \pi_{p+1}) \subseteq F \times G$ of vertex correspondences. A vertex correspondence is a pair $\pi = (f, g)$ with $f \in F$ and $g \in G$, denoting that

ing on the configuration, it is a weakly connected polygon (b) possibly even featuring dangling outgrowths (c)

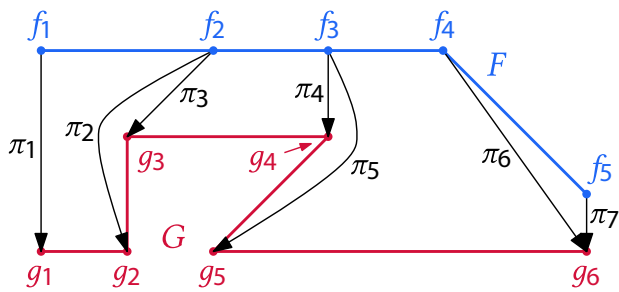


Fig. 4 A correspondence between polylines F and G fulfilling Requirements (R1) – (R3) and thus being a schematic polyline correspondence. Every vertex of F and G is part of at least one vertex correspondence (black arrows)

vertex f corresponds to vertex g . We call f the *source* and g the *target* of the correspondence π . Let the trace $\text{tr}(\pi)$ of a vertex correspondence $\pi = (f, g)$ be the geodesic path from f to g inside the morph region R . We call Π a *schematic polyline correspondence* if it fulfills:

(R1) **MONOTONE**: For the correspondences $\pi' = (f_i, g_j)$ and $\pi'' = (f_k, g_l)$ in the sequence Π with $i < k$ it holds that $j \leq l$.

(R2) **COMPLETE**: All vertices of F and G are part of at least one vertex correspondence in Π .

(R3) **ORIENTATION- PRESERVING**: If two consecutive vertex correspondences span a segment on both input polylines, i.e., $\pi_k = (f_i, g_j)$, $\pi_{k+1} = (f_{i+1}, g_{j+1}) \in \Pi$, then these segments f_{i+1}^{i+1} of F and g_{j+1}^{j+1} of G have the same direction.

Figure 4 gives an example of a schematic polyline correspondence.

Operations Let an *operation* o be a pair (π_i, π_{i+1}) of two consecutive vertex correspondences in Π . Requirements (R1) and (R2) imply that given the vertex correspondence $\pi_i = (f_j, g_k) \in \Pi$, there are at most three valid candidates for the succeeding vertex correspondence $\pi_{i+1} \in \Pi$; see Fig. 5:

$\pi_{i+1} = (f_{r+1}, g_{c+1})$ resulting in a segment-to-segment correspondence of f_r^{r+1} to g_c^{c+1} . We call the pair (π_i, π_{i+1}) a **Match** operation.

$\pi_{i+1} = (f_r, g_{c+1})$ resulting in a vertex-to-segment correspondence of f_r to g_c^{c+1} . A new segment appears during the morph. We call the pair (π_i, π_{i+1}) an **Insert** operation.

$\pi_{i+1} = (f_{r+1}, g_c)$ resulting in a segment-to-vertex correspondence of f_r^{r+1} to g_c . The segment disappears during the morph. We call the pair (π_i, π_{i+1}) a **Delete** operation.

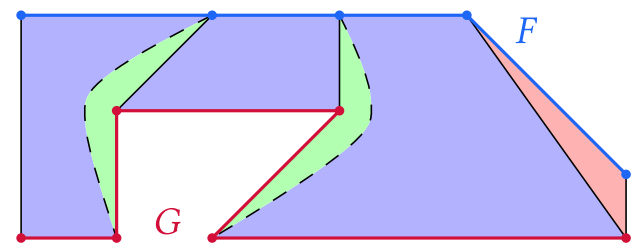


Fig. 5 The operations resulting from the correspondence displayed in Fig. 4. Segments of equal orientation can form a **Match** (blue). Otherwise, segments either form an **Insert** (green) or a **Delete** (red). The red lines indicate vertex traces. The dash-dotted traces are displaced to allow distinguishing all operations

We denote the first vertex correspondence in an operation o with $\pi_L(o)$ and the second one with $\pi_R(o)$. Analogously, $\text{tr}_L(o)$ and $\text{tr}_R(o)$ are the traces of these vertex correspondences. An operation describes the correspondence between a vertex or segment on F to a vertex or segment on G . We denote the vertex or segment on F as $\text{source}(o)$ and the vertex or segment of G as $\text{target}(o)$.

Requirement (R3) enforces that **Match** operations are only valid between segments of equal orientation, i.e., both polyline segments that are part of the **Match** have the same direction. We denote this direction with $\text{dir}(o)$. During the morph, the $\text{source}(o)$ for every segment $o \in \Omega$ will be transitioned parallelly toward $\text{target}(o)$. Thus, every intermediate polyline has the same direction $\text{dir}(o)$ as the input polylines within the operation.

Properties of Schematic Polyline Correspondences Due to the monotonicity of the correspondence, the individual vertex traces, defined as geodesic paths between corresponding vertices, do not cross each other. Furthermore, a geodesic path is inherently self-intersection-free. Consequently, the vertex traces partition the morph region R into distinct subregions, with each subregion corresponding to an operation, for example, Fig. 5. This ensures three crucial properties. Firstly, each operation involves the movement of a single segment, which can proceed in parallel to its original position, thereby preserving the schematic representation of the polylines throughout the transition, illustrated in Fig. 6. Secondly, a single segment cannot have self-intersection; thus, each operation transition is self-intersection-free. Thirdly, as the subregions defined by the vertex traces are a tessellation of the morph region, they are mutually disjoint. Since each segment remains confined within its operations' subregion, no two segments of different operations can interact during the morph. Put together, these three properties guarantee that the correspondence generated by our method results in a morph that remains schematic, self-intersection-free, and contained within the morph region (which we call self-contained).

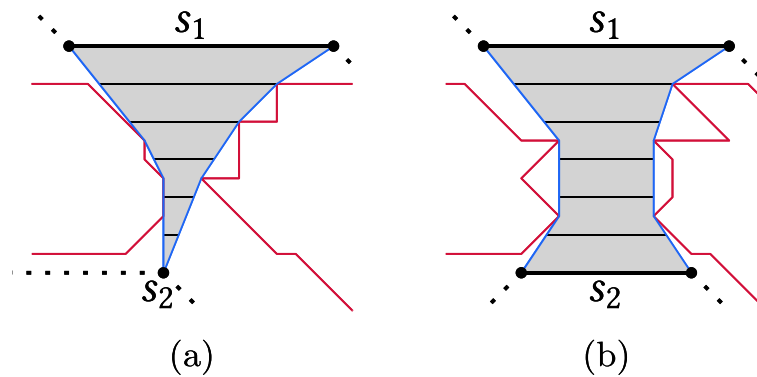


Fig. 6 Illustration of correspondence properties by taking the example of an **a** Delete and **b** Match operation. The blue lines are the vertex traces between s_1 and s_2 , which avoid crossing any other parts of the input polylines not involved in the correspondence (red). These traces

induce the gray subregion of the morph region. Within this subregion, the segment (black) is translated parallelly. That way, it is ensured that the morph is schematic, self-intersection-free, and contained within the morph region (i.e., self-contained)

Optimization The correspondence needs to be optimized to generate a comprehensive morph animation. Less movement leads to less complex morph animations; thus, we minimize the distance the vertices move during the morph. Hence, we want to solve the following optimization problem:

Problem 3.1 (Optimal Correspondence) Given two simple schematized polylines $F = (f_1, \dots, f_n)$ and $G = (g_1, \dots, g_m)$ that do not intersect, find the correspondence $\Omega = (o_1, \dots, o_p)$ that fulfills Requirements (R1)-(R3) and that minimizes a weighted sum of the vertex traces' lengths.

Algorithm

In the following, we discuss an efficient algorithm to solve *Optimal Correspondence*. We are looking for the vertex-based correspondence between the two polylines F and G that minimizes the weighted sum of the vertex traces' lengths. Nöllenburg et al. (2008) introduced a dynamic programming approach for solving the optimal correspondence for a generic additive cost function c . However, their approach does not guarantee that the generated correspondence results in a schematic morph. That is, Requirement (R3) is not fulfilled. In their algorithm, a dynamic programming table T is used, where every table entry $T[r, c]$ stores the total cost of morphing subpolyline f_1^r onto g_1^c . The table is filled recursively, starting from the initial solution $T[0, 0] = 0$. Each table entry's value is the sum of the previous entry and the cost of the applied operation. To solve Problem 3.1, we need to adapt this update step by (a) introducing an appropriate cost function for the operations and (b) guaranteeing that Requirement (R3): ORIENTATION- PRESERVING is fulfilled.

Operation Costs The cost of an operation o is defined by the cost function $c: o \rightarrow \mathbb{R}$. We want to minimize the distance each vertex travels during the morph. Therefore, we consider the sum of the vertex traces' length per operation as the cost c^* :

$$c^*(o) = \ell(\text{tr}_L(o)) + \ell(\text{tr}_R(o)). \quad (1)$$

Intuitively, an operation spanning a large section of the input polylines should have a higher importance regarding the final correspondence than an operation covering only a tiny part of the polylines. Therefore, we weight the cost with a weighting term $w(o)$ proportional to the length of the covered polyline segments:

$$w(o) = \ell(\text{source}(o)) + \ell(\text{target}(o)). \quad (2)$$

Each segment of the input polylines F and G is part of precisely one operation in Ω . As a result, it is easy to show that $\sum_{o \in \Omega} w(o) = \ell(F) + \ell(G)$, which we use as a normalization term η . Thus, the final cost c_{opt} for an operation is given by:

$$c_{\text{opt}}(o) = \frac{w(o)}{\eta} \cdot c^*(o). \quad (3)$$

On the one hand, the weighting ensures that operations involving longer sections of the input polylines are more important in the correspondence. The weighting ensures that the cost is independent of the level of discretization of the input.

We can compute $T[r][c]$ for $r, c > 0$ by using the optimal solution for the subpolylines defined by the three possible operations:

$$T[r][c] = \min \begin{cases} T[r-1, c] + c_{\text{opt}}(o_D), & \text{Delete} \\ T[r-1, c-1] + c'_{\text{opt}}(o_M), & \text{Match} \\ T[r, c-1] + c_{\text{opt}}(o_I), & \text{Insert} \end{cases} \quad (4)$$

Guaranteeing Schematization To guarantee Requirement (R3), the cost function c_{opt} introduced in the last section is slightly modified for `Match` operations. In the case that a `Match` o_M is invalid because its source and target have different orientations, infinity is returned:

$$c'_{\text{opt}}(o_M) = \begin{cases} c_{\text{opt}}(o_M), & \text{if } \text{dir}(\text{source}(o_M)) = \text{dir}(\text{target}(o_M)) \\ \infty, & \text{else.} \end{cases} \quad (5)$$

Complexity The dynamic programming table has size $n \times m$ where n and m are the numbers of points in F and G , respectively. This results in a running time of $\mathcal{O}(nm \cdot C)$ with C denoting the time needed to evaluate the cost function c_{opt} . Precomputing the geodesic paths between the input polylines allows us to have constant evaluation times for c_{opt} (i.e., $C \in \mathcal{O}(1)$) resulting in a complexity of $\mathcal{O}(nm)$.

Timing

Until now, we have considered the requirements of having a (1) schematic, (2) self-intersection-free, and (3) self-contained morph. In the timing problem, we consider the requirement for a constant velocity of the moving segments in the morph.

In contrast to the geometric shape of the point movement, the velocity of a point along its trace has yet to be defined. To model this temporal component, we extend the traces for a vertex correspondence from point $f \in F$ to point $g \in G$ by a time parameter t . To make the transition as smooth as possible and thus reduce the visual complexity, we require that each segment has a constant velocity in the direction of its normal while it is moving. However, to fulfill the requirement of parallel movement, segments can start and end their movement later than the full morph. Figure 7 shows the timing for the correspondence displayed in Fig. 5. Each operation o_i (i.e., pair of consecutive vertex correspondences) defines a transition of one segment s_i from the source to the target polyline. We denote the subpolyline of M between its vertices i and j at time t with $m_{ij}(t)$.

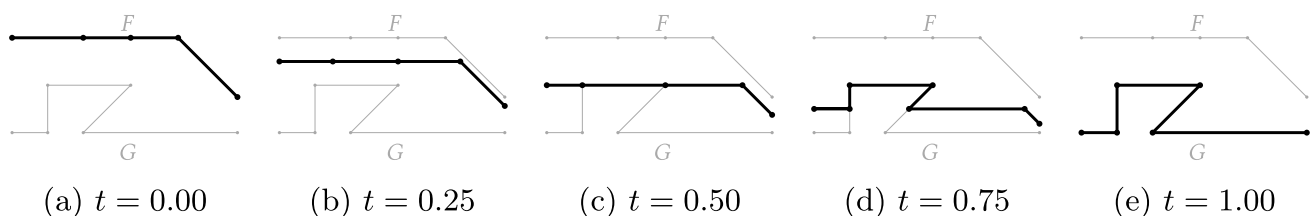


Fig. 7 The morph resulting from the correspondence shown in Fig. 5 is displayed. Depicted are five timesteps of the animation. At $t = 0.50$, the central segment has reached its destination while the others continue

Requirements

We impose two requirements that influence the timing. First, to preserve the schematization, each polyline segment s can only be moved parallelly during the morph. Secondly, to reduce the visual complexity of the morph, each segment should morph with a constant velocity. More specifically, a segment can start its movement later and end its movement earlier than the full morph. Still, the segment's velocity in its orthogonal direction is constant during this movement. This means that each segment s has a movement interval $I_s = [t_{\text{start}}, t_{\text{end}}] \subseteq [0, 1]$.

We solve the timing in a two-step process. Firstly, we partition the morph region R into a set of *transition cells*. In each transition cell, every segment starts and ends its movement simultaneously. Secondly, we determine the starting and ending times of the individual transition cells by setting up and solving a system of linear equations that guarantees temporal consistency between the cells.

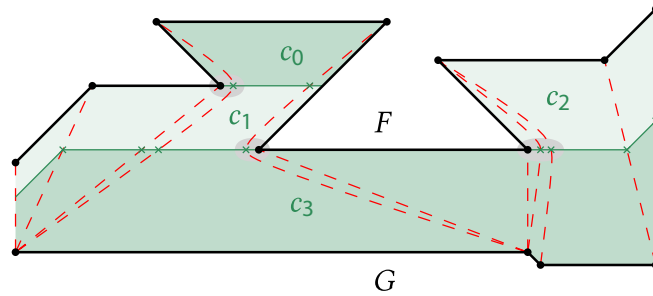
Creating Transition Cells

A transition cell c is a subregion of R with the property that all vertex traces passing through the cell are straight lines. This allows us to apply uniform motion to the vertexes inside a single cell. By the intercept theorem, this uniform motion on straight-line traces results in a parallel movement of the transitioning polyline segments. The transition cells of the morph form a tessellation of the morph region R .

By definition, a region will be separated into two transition cells at bends in a vertex trace. Therefore, we iteratively explore all operations in the given correspondence. For every bend in $\text{tr}_L(o_i)$, i.e., the operation's first trace, the bend is propagated forward through the remaining operations by casting a ray in the operation's direction $\text{dir}(o_i)$. For every bend in $\text{tr}_R(o_i)$, i.e., the operation's second trace, the bend is propagated backward through the previous operations by casting a ray in the opposite direction of $\text{dir}(o_i)$. The lines created that way are collected in a set $\mathcal{L}$ used to split the morph region R into the transition cells. Figure 8 gives an example of a decomposition into transition cells. Lines in $\mathcal{L}$

moving. Due to the constant velocity constraint, the segments left and right of the stopped segment have the same velocity

Fig. 8 Creation of transition cells. Vertex traces (red) that are colinear are displaced for visualization purposes. At bends in the vertex traces, rays (green) are shot parallel to the operation's segments. These rays divide the morph region into transition cells (green)



are shown in green. They decompose the morph region in four transition cells: c_0 , c_1 , c_2 , and c_3 .

Transition Timing

Each transition call c spans a sequence of operations $O = (o_i, \dots, o_j)$ with $i \leq j$ and has a starting time $t_s(c)$ and an ending time $t_e(c)$, with $0 \leq t_s(c) < t_e(c) \leq 1$. Thus, a transition cell c describes the morph from $m_{ij}(t_s(c))$ to $m_{ij}(t_e(c))$. We call $m_{ij}(t_s(c))$ the input of the transition call and $m_{ij}(t_e(c))$ its output. Between $t_s(c)$ and $t_e(c)$, all input segments of c move at a constant velocity toward the output segments of c .

Constraints Some segments move through several transition cells. We call such segments *cell-linking*. Cell-linking segments impose two constraints on the transition cells' starting and ending times. Firstly, the segment should not stop at the cell border. Therefore, the ending time of the first cell must be the same as the starting time of the second cell. Take the leftmost segment in Fig. 8 as an example. It starts its movement in c_1 and continues through c_3 . Here, c_3 must start its movement when c_1 ends moving. Analogously, c_0 and c_1 as well as c_2 and c_3 are linked. Secondly, segments must have constant velocity throughout the morph, interlinking the transition times of all cells they pass through.

Transition Cell Graph We observe that we can arrange the set of transition cells $C = (c_1, \dots, c_k)$ in a directed graph $\mathcal{T} = (C, E)$, with the transition cells being the graph's vertices. Two neighboring transition cells, i.e., transition cells separated by a line $l \in \mathcal{L}$, are connected by a directed edge $e \in E$. The direction of the edge e is defined by the vertex correspondences. The morph region R is simple, meaning it has no holes and is connected. Therefore, the transition graph $\mathcal{T}$ does not contain loops and consists of a single connected component, resulting in a graph with $|C| = k$ vertices and $|E| = k - 1$ edges.

Computing Cell Starting and Ending Times We solve the timing using a system of linear equations with the starting and ending times of the cells as unknowns. Thus, each vertex of the transition graph $\mathcal{T}$ yields two unknowns, resulting in $2k$

unknowns. Additionally, each edge $e = (c_i, c_j)$ in $\mathcal{T}$ imposes two constraints. Firstly, the starting and ending times of c_i and c_j must be consistent:

$$t_e(c_i) = t_s(c_j) \quad (6)$$

Secondly, the pair of transition cells are separated by at least one segment s_b that is part of the output of c_i and the input of c_j . The orthogonal velocity of s_b must be the same in both cells. We denote the orthogonal distance the segment s_b travels inside cell c_i with d_{ib} . The orthogonal velocity of s_b in c_i is given by

$$v_b = \frac{d_{ib}}{t_e(c_i) - t_s(c_i)}. \quad (7)$$

Thus, the second constraint induced by e is

$$\frac{d_{ib}}{t_e(c_i) - t_s(c_i)} = \frac{d_{jb}}{t_e(c_j) - t_s(c_j)} \quad (8)$$

To yield a linear constraint in the unknowns, we take the reciprocal of the equation. Further, to improve the numerical properties of our system of linear equations, we normalize the constraint with $\eta = \min(d_{ib}, d_{jb})$:

$$\eta \cdot \frac{t_e(c_i) - t_s(c_i)}{d_{ib}} = \eta \cdot \frac{t_e(c_j) - t_s(c_j)}{d_{jb}} \quad (9)$$

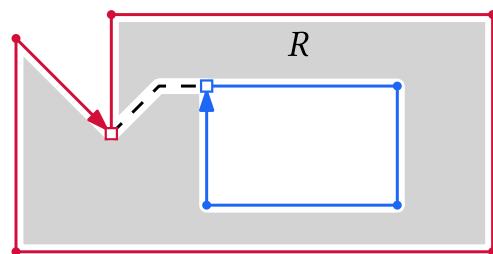


Fig. 9 Polygons preprocessing. The polygon outlines are cut open at two cut locations (squared vertices). A schematized cut line connecting the cut locations splits the morph region R (gray, slightly buffered to the inside for visualization purposes)

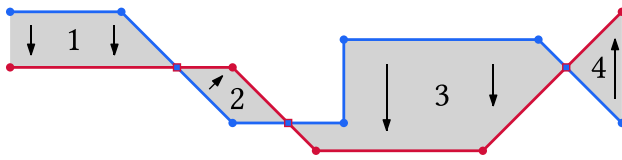


Fig. 10 For intersecting input polylines, the morph is split into subinstances at the polyline intersections

Introducing a Datum The system of equations consists of $2k$ unknowns. Each edge $e \in E$ yields two linearly independent constraints, resulting in $2(k-1)$ constraints. As a consequence, we have a rank deficiency of 2. This is because the computed times can be scaled and offset arbitrarily with the current definition. To deal with this, we introduce a datum by fixing the starting and ending times of an arbitrary cell c to $t_s(c) = 0$ and $t_e(c) = 1$. After solving the system of linear equations, we rescale all computed times to the interval of $[0, 1]$.

General Case

Until now, we have only considered the base case, in which we assume that (i) our input polylines do not intersect and (ii) the two geodesic paths connecting their endpoints do not intersect. These assumptions on our input do not always hold. Additionally, F and G might form the outlines of polygons. We can apply simple preprocessing steps to fulfill these two requirements in almost all practically relevant situations.

When handling polygon outlines, we divide them at designated *cut locations* to obtain two polylines. To generate the morph region, we connect these cut locations with a schematized line, referred to as *cut line* (e.g., using the algorithm by Speckmann and Verbeek 2018), see Fig. 9. If the polygon outlines intersect, one of their intersection points serves as a natural cut location for both outlines, making the computation of the cut line superfluous. Otherwise, a simple approach to selecting the cut locations is by using the pair of vertices between the two polygons closest to each other.

For (i), we split the polylines at their intersection points. If the intersection points occur in the same order along the

two polylines, we create one morph subinstance per pair of subpolylines, see Fig. 10. Each subinstance is solved independently. In our experiments with two polylines F and G corresponding to the boundaries of simple polygons, however, we encountered a few examples where the order of the intersection points differed for F and G . We handle this case by computing the intersection of the polygons enclosed by F and G . From the resulting multi-polygon, we select the largest component C . Then, we compute a morph between polyline F and the boundary ∂C of C and a morph between ∂C and polyline G . An example of this situation is given in Fig. 11.

The most challenging case is that F and G do not correspond to the boundaries of polygons, and the order of their intersection points differ for F and G . This scenario is displayed in Fig. 12. This case never occurred in our experiments, and we leave it as a challenge for future research. However, we think computing an intermediate polyline and performing the morph in two steps (as in our approach for polygonal rings) is promising.

Finally, Fig. 13 shows an example where (ii) is violated. We can reverse the target polyline for these cases to set up the morph region (Fig. 13b).

Evaluation

In this section, we evaluate our morphing algorithm. The evaluation can be separated into three main parts. First, we perform a quantitative evaluation of our approach by comparing it to morphs generated with a baseline approach that does not guarantee schematization (SCHEMATIC) during the morph or topological correctness (i.e., SELF-INTERSECTION-FREE and SELF-CONTAINED). These baseline methods closely resemble the morph algorithm introduced by Nöllenburg et al. (2008), a state-of-the-art technique for generating morphs in cartography. While there are minor differences, the core principles remain largely the same, enabling a meaningful comparative evaluation against state-of-the-art methods. Specifically, we want to analyze how strongly these constraints influence the optimization function c_{opt} of the morph.

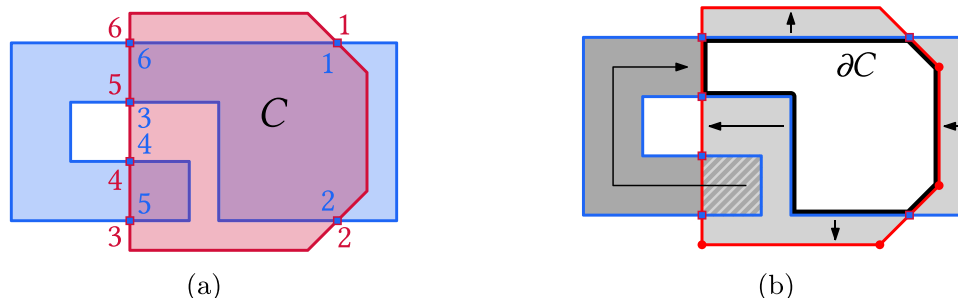


Fig. 11 **a** The intersections of the blue and the red polygon occur in different orders around the polygon outlines (numbers indicate the sequence). **b** The morph is computed toward the outline ∂C of the largest component C of the arrangement

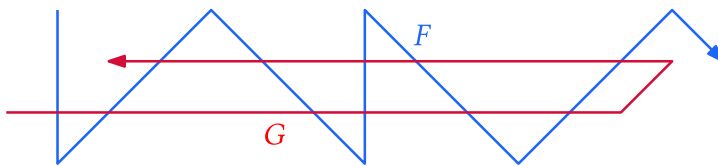


Fig. 12 Illustration of the only case not (yet) covered by our approach. The input polylines do not form the outline of a polygon, and their intersection points occur in different orders along the two polylines. This case never occurred in our experiments

In the second step, we perform a qualitative analysis by closely examining generated morphs. These first two steps are performed on closed input polylines. In the third step, we apply our algorithm to non-closed polylines, each representing a different level of schematization for the same input feature, to evaluate its effectiveness in the context of continuous generalization.

Evaluation Instances For the quantitative evaluation, we use *isochrones* generated for the cycling network of Bonn, Germany. Specifically, we use isochrones generated for different cyclist routing profiles. For this, we use the routing model introduced by Forsch et al. (2023) to calculate the accessibility for cyclists. In their model, a balance factor α is introduced that expresses the cyclists' routing preferences. When applied to the usage of cycleways, an $\alpha > 0.5$ indicates preference of these designated cycleways while an $\alpha < 0.5$ indicates an avoidance of these paths. For $\alpha = 0.5$ the cyclist is indifferent on the type of path and only minimized the geometric length. Using this routing model, the generated isochrones will display the locations that are reachable within a given maximum amount of *effort* concerning the routing profile. We generate octilinear isochrones using the visualization component described by Forsch et al. (2021).

We generate isochrones for three different starting locations in Bonn. We apply $k = 6$ different maximum efforts for each starting location for two prevalent routing profiles identified by Forsch et al. (2023): cyclists sticking to shortest paths (CON, $\alpha = 0.45$) and cyclists wanting to stick to sign-posted cycleway (PRO, $\alpha = 0.65$). If the isochrones have holes, we remove them, taking only the outer ring as input to our algorithm. We use these isochrones to generate two sets of morph instances per starting location:

1. **Max Effort:** For all k maximum efforts, we compute the morph to all larger values of maximum efforts, resulting in $\frac{(k-1)k}{2}$ instances per routing profile. An isochrone of smaller maximum effort is contained inside an isochrone of larger maximum effort, making the Self-Contained property crucial for interpretation.
2. **Routing Profile:** For all k maximum efforts, we compute the morph between the two routing profiles.

Isochrones in this group often intersect, as one routing profile might reach further than the other in one area but shorter in another.

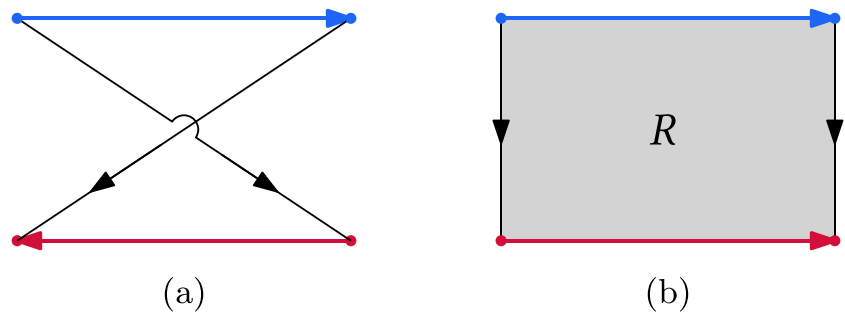
These two groups result in 21 morph instances per starting location and 63 morph instances. As described in the “**Base Case: Morphing Simple Polylines**” section, the input polylines are split at intersection points, and a morph is generated individually for each subinstance.

Implementation To evaluate our approach, we implemented it as a single-threaded Java application. The implementation makes use of two open-source libraries, namely GeoTools (Turton 2008) for handling geographic data and JGraphT (Michail et al. 2020) for the graph structures and algorithms. The implementation, as well as the evaluation instances, are published as open source.¹ The experiments are executed in a Java 11 Runtime Environment on an AMD Ryzen 7 processor clocked at 2.3 MHz.

Baseline Solution We want to evaluate the influence of the constraints SCHEMATIC, SELF-CONTAINED, and SELF-INTERSECTION-FREE on the optimization criterion of the morph. For this, we define the *baseline* solution as a solution that does not fulfill these properties. This baseline solution can be generated using the solution to the correspondence problem (“**Correspondence**” section) with straight-line vertex traces and uniform motion (i.e., constant vertex velocity) along these traces. This approach closely resembles the morphing procedure introduced by Nöllenburg et al. (2008) with slight differences in the applied cost function. Additionally to the baseline solution, we introduce the *schematic baseline* solution that fulfills the SCHEMATIC property. We use straight-line traces as the baseline solution for this, but we use the modified cost function c_{opt}^* instead of c_{opt} . This solution is a trade-off between the baseline solution and our approach when topological correctness is not strictly needed.

¹ <https://gitlab.igg.uni-bonn.de/rukemn/traveltimemaps>.

Fig. 13 If the geodesic paths between the source and target (black) cross (a), the target polyline is reversed to create the morph region R (b)



Quantitative Evaluation

Splitting the 63 morph instances at the intersection of their input polylines resulted in 226 subinstances. For each subinstance, we generated a morph using the baseline, the schematic baseline, and our approach. Our primary focus is to analyze how much the constraints we induce on the morphing impact the total cost c_{opt} of the morph. The results of our quantitative analysis are shown in Table 1. When comparing the schematic baseline to the baseline, the cost only increases by less than 5%. Remember that the difference between c_{opt}^* used in the schematic baseline to c_{opt} used in the baseline solution is that matches are only allowed between segments of equal orientation in c_{opt}^* . Thus, this increase is explained by using more Insert and Delete operations instead of Matches. When comparing the schematic baseline to our approach, the increase in the cost is only 0.5%. We conclude that guaranteeing the schematization has the largest influence on the morph cost, while guaranteeing the topological correctness has only a minor effect.

We need to examine the profit gained to assess whether the increase in the optimization target is reasonable. Table 2 gives an overview of how many subinstances fulfill the properties we require in our approach. By construction, all properties are fulfilled in every instance for our approach. Self-intersections only occur in $\approx 4\%$ of the subinstances for the baseline approaches, which might lead to the conclusion that they are only a minor issue. In contrast, the self-contained property is only fulfilled in $\approx 30\%$ of the instances for the two baseline approaches. As a result, the slight increase in our

optimization target seems warranted for applications where the topological correctness of the morph is needed.

Running Times Figure 14 shows the running time of the morph based on the used approach. The two baseline approaches use straight-line traces with uniform motion. Therefore, once the correspondence is known, the morph is completely defined. For all approaches, our implementation takes under three seconds to solve the correspondence, even for larger instances.

Our approach uses geodesic paths, which we precompute using shortest-path queries in a visibility graph. Our naive approach for computing the visibility graph takes up to 25 s for larger instances, making up the largest part of the algorithm running time. Solving the timing takes approximately as long as solving the correspondence, with running times smaller than 6 s. In conclusion, making the precomputation of geodesics more efficient is mandatory for real-world applications.

Qualitative Evaluation

After analyzing the approach’s impact on our optimization target, we perform a qualitative analysis based on an example. Figure 15 displays an intermediate polyline of the morph between two octilinear isochrones generated for two different values of maximum effort. The morph is generated with the baseline approach (left) and our approach (right). The

Table 1 Results of the quantitative analysis

	Baseline	Schematic baseline	Our approach
Increase in c_{opt}	Reference	+4.7%	+5.2%
Given is the relative increase of the morph cost c_{opt} for our approach and the schematic baseline compared to the baseline			

Table 2 Properties of the resulting morph when using the three approaches

	Baseline	Schematic baseline	Our approach
Self-contained	30.1%	30.1%	100%
Self-crossing-free	97.4%	96.0%	100%
Schematic	0.0%	100.0%	100%

Given is the percentage of subinstances that fulfill the specific property

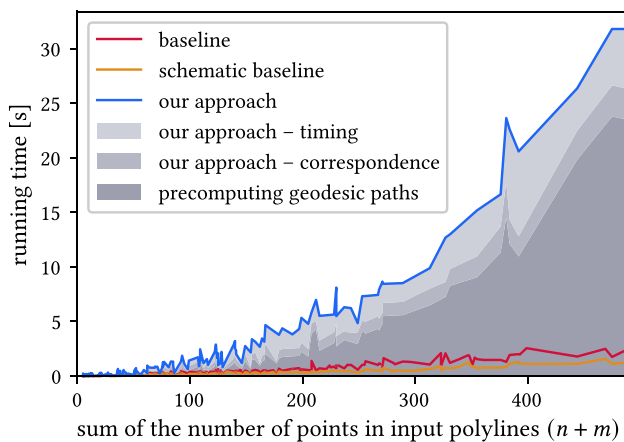


Fig. 14 Running times of the morphing depending on the input size

morph region and the source and target polyline are shown in gray.

For the baseline solution, many segments of the intermediate polylines do not follow the octilinear directions (red arrows highlight some of them). Additionally, the intermediate polyline leaves the morph region (see red highlights). Such incidents limit the morph's readability, especially for cases like the one displayed, where the source polyline is contained inside the target polyline. They indicate that one can reach further between the two isochrones.

In contrast, the solution generated with our approach is self-contained and schematic. Nonetheless, the intermediate polylines do not form a closed line anymore. The reason for this lies in the timing of the morph. Recall that our approach is initially designed for polylines, not polygonal rings. To use the algorithm for polygonal rings, we cut the rings open at a vertex and apply the morphing to the resulting polyline. The

Table 3 Properties of the resulting morph when using the three approaches

	Baseline	Schematic baseline	Our approach
Self-contained	66.1%	63.5%	100%
Self-crossing-free	100.0%	100.0%	100%
Schematic	0.0%	100.0%	100%
Increase in c_{opt}	Reference	+3.7%	+3.8%

Given is the percentage of substances that fulfill the specific property and the relative increase of the morph cost c_{opt} for our approach and the schematic baseline compared to the baseline

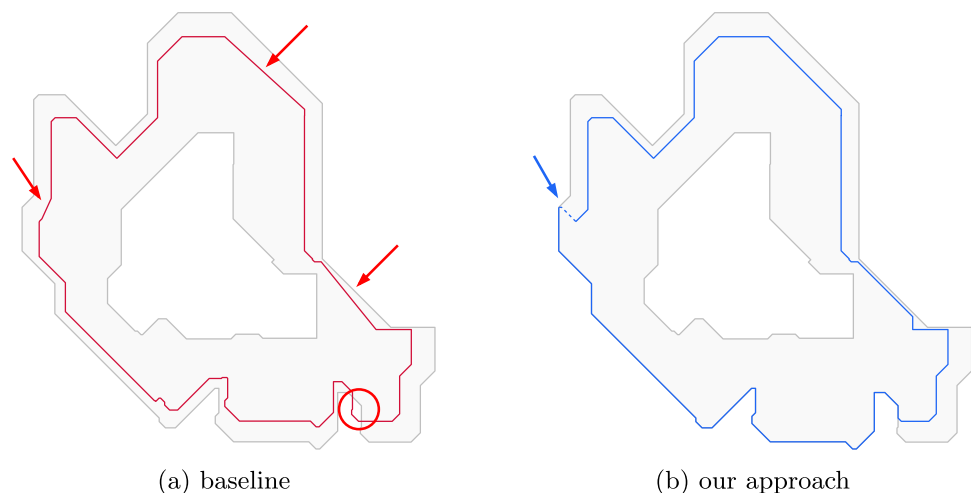
algorithm for computing the transition timing ensures consistency within the morph, but a loop closure for rings is not yet considered. A simple approach to this issue is introducing a loop closing segment (dotted line in Fig. 15b). This closing segment will always be on the cut line, which is schematized by construction. Thus, the resulting loop closing segment also follows the schematization.

Continuous Generalization

As discussed in the “[Related Work](#)” section, continuous generalization is a key application of morphing in cartography. To assess the suitability of our approach for this purpose, we focus on the example of rivers. Specifically, we evaluate the effectiveness of our method in supporting continuous generalization within the context of schematization by morphing between schematized versions of rivers with varying levels of abstraction.

Input We generated the input instances for 10 European rivers – Danube, Ebro, Elbe, Loire, Po, Rhine, Rhone, Seine, Thames, and Vistula – whose geometry we extracted

Fig. 15 Morph between two isochrones for different maximum effort values. Shown is the intermediate polyline for $t = 0.8$ generated using the baseline and our approach. **a** The baseline approach is not schematic (arrows) and can move outside the morph region (circle). **b** For the loop closure, a schematic path between the start and end of the intermediate polylines is added in our approach (arrow)



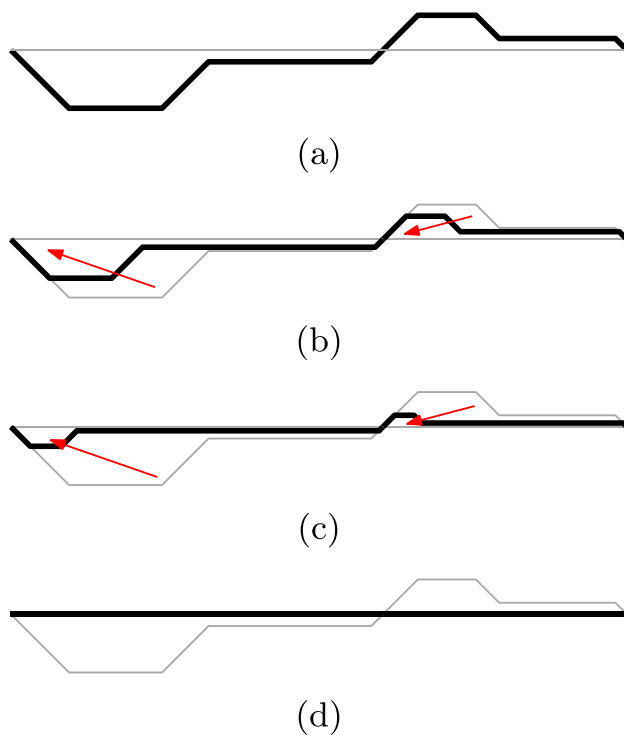


Fig. 16 The last two morph instances for the River Rhine. Using only basic operations, the convexities collapse onto one of the cutting points

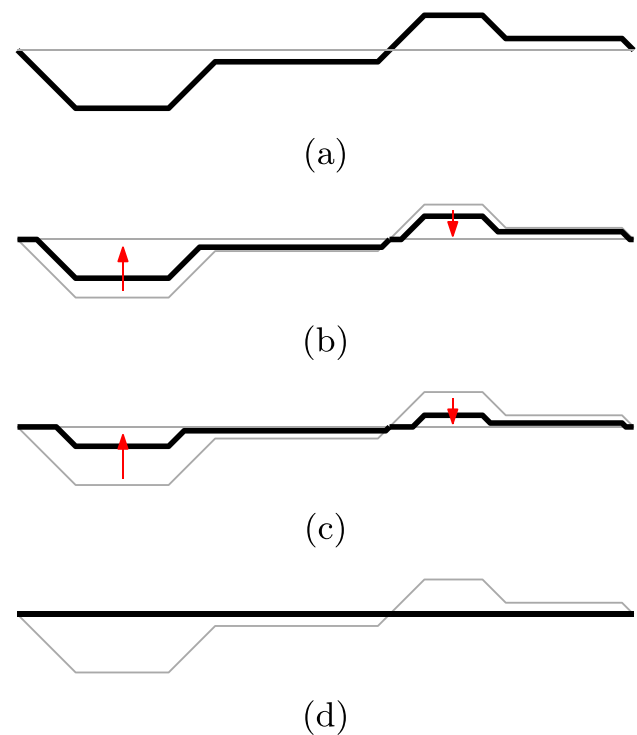


Fig. 17 The last two morph instances for the River Rhine. When multi-operations are allowed, the morph appears more natural

from OpenStreetMap.² For generating schematized variants of the geometries, we apply a grid-based algorithm, detailed in Appendix 4. For each river, we produced two schematized versions with differing levels of abstraction. We then generated a morphing sequence for each pair, transitioning from the more schematized version to the less schematized one, resulting in ten morphs – one for each river. The resulting morphs are found on <https://www.geoinfo.uni-bonn.de/morphing>.

Quantitative Evaluation Since the morph inputs are generated for the same geometric feature, they contain many intersections. After applying the preprocessing steps — specifically, dividing the inputs into subpolylines as illustrated in Fig. 10) — we end up with 189 morph subinstances, averaging about 19 subinstances per river. Due to the large number of subinstances, each individual subinstance is relatively small, with the largest containing only 32 vertices on the source polyline. When conducting the quantitative evaluation, similar to the method used for isolines, we observed that these smaller instances exhibit slightly different behavior than the isolines. The results of this evaluation are presented in Table 3.

In this scenario, none of the methods introduced self-intersections during the morph, and nearly two-thirds of all subinstances remained self-contained, even when using the baseline methods. Computation times were also much faster, with the slowest instance completing in less than 0.3 s, including geodesic path computations. When comparing the schematic baseline solution to our approach, the differences are minimal. As such, the additional complexity introduced by computing geodesic paths might not be warranted given the scenario of continuous generalization, especially since the self-contained property appears less critical for rivers than it is for isolines.

Qualitative Evaluation When examining the generated morphs, we observe that many points of the less abstracted source polyline seem to collapse onto a single point on the more abstracted target polyline. Figure 16 illustrates this effect with two subinstances of the River Rhine where the convexities of the source polyline collapse to the left, converging onto a single vertex of the target polyline.

The reason for this is that the current approach only allows basic operations, which are a one-to-one correspondence between elements on the source and target polyline. In the example where the target polyline consists of a single element, no other correspondences are possible. To address this limitation, multi-operations could be introduced,

² www.openstreetmap.org.

enabling one-to-many and many-to-one correspondences. Figure 17 shows the morphing for the same subinstances multi-operations are enabled. In this scenario, the convexities collapse more naturally, aligning with the nearest subsection of the target polyline.

Conclusion and Future Work

We have presented and evaluated an approach for generating a schematic morph animation for *C*-oriented input polylines. The resulting animation fulfills several constraints imposed to reduce its visual complexity and thus improve its readability. In particular, the animation is *C*-oriented, self-intersection-free, self-contained in the morph region, and has a constant velocity. While the algorithm is designed for polylines, we showed that it can also be applied to polygonal shapes and yields promising results. Nonetheless, we identified several areas where improvements to the algorithm can be made. In the following, we review possible directions for future work.

Multi Operations We defined three basic operations, each matching single segments. In cases where the resolution of the source and target polylines is very different, this might lead to suboptimal morphs. To counteract this, *multi-operations* that can match a whole sequence of segments on one polyline to a segment of the other polyline can be introduced. Our current implementation already supports these multi-operations, but a more detailed evaluation of this extension is needed.

Finding Shape Correspondences for Morphing Multi-Shapes In this article, we assumed that we are given two polylines that need to be morphed onto each other. For practical application, it might not always be clear which shapes need to be morphed onto each other. For example, isochrones generated for the public transport network might consist of multiple polygons. For these applications, methods for retrieving the input instances for morphing need to be explored. One approach to address this problem is *bipartite matching*. This method involves modeling the two sets of polygons as vertices in a bipartite graph, with edges representing the suitability of matches between them. While bipartite matching can effectively handle one-to-one correspondences, where one polygon from the first set is morphed to one polygon in the second set, many real-world scenarios require one-to-many or many-to-many matches. Therefore, extending the approach to accommodate these more complex relationships is essential for practical applications.

Utilizing Different Cost Functions This article minimized the weighted distance the points move during the morph. While this simple metric already yields promising results, other metrics might improve the morph. For example, these metrics could take shape characteristics or prior knowledge on the input into account.

Improving Running Times The analysis of the running times showed that the computation of the geodesic paths is the bottleneck in our approach. Although this article has not focused on the efficient computation of geodesic paths, it can significantly improve the approach's utility. This problem has received much attention in the computational geometry community, and efficient algorithms exist to solve this problem. An example is the algorithm published by Guibas and Hershberger (1989) that allows computing the length of the shortest path inside a simple polygon with n vertices in $\mathcal{O}(\log n)$ time based on a triangulation and a hierarchical decomposition of the polygon. Utilizing such a specialized algorithm could substantially improve the running times of our algorithm.

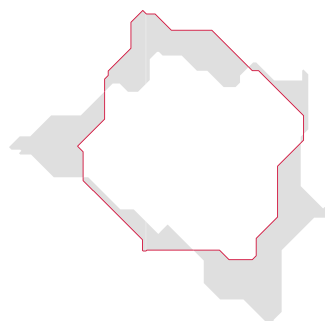
Additionally, parallel computing can be employed to significantly speed up the computation of geodesic paths. The running times recorded in the evaluation section are based on a single-core implementation. However, because each geodesic path is independent of the others, implementing the computations in a multi-core environment is straightforward, allowing for speed-ups that match the number of available cores.

Relaxing the Constant Velocity Constraint We showed that the requirements we impose on the morphing (schematic, self-intersection-free, self-contained, and constant velocity) are mutually compatible and can be fulfilled without conflicting with each other. However, they uniquely define the morph animation, leaving no further degree of freedom for additional optimization targets. To introduce some flexibility into the morphing process, the constant velocity constraint could be relaxed. For example, allowing slight variations in speed could help eliminate the need for closing segments while still aiming to minimize speed fluctuations.

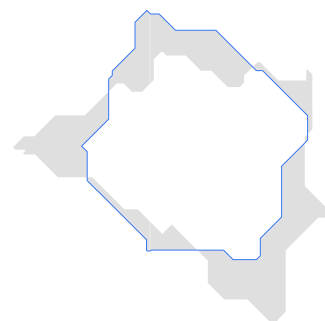
User Evaluation The quantitative evaluation demonstrates that our algorithm performs comparably to state-of-the-art approaches for morphing in cartography. However, this assessment is limited by the absence of user feedback and usability data in real-world applications. To thoroughly evaluate the practical effectiveness and user acceptance of our method in generating meaningful morphs, it is essential to conduct a comprehensive user study.

Appendix 1. Isochrones — Routing Profiles

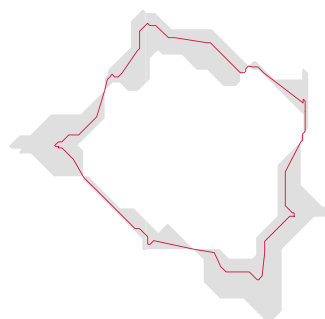
Fig. 18 Morph between two isochrones generated for different routing profiles. The morph consists of six subinstances. Baseline (left) and our approach (right)



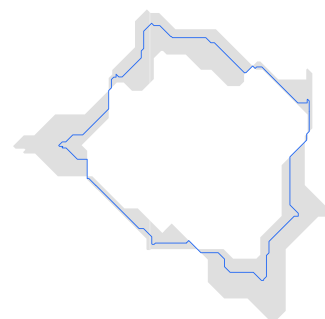
(a) $t = 0.00$



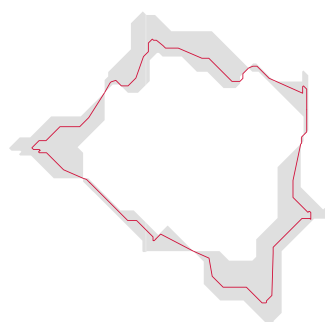
(b) $t = 0.00$



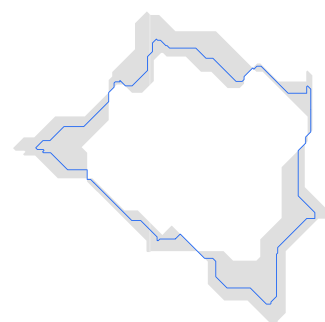
(c) $t = 0.33$



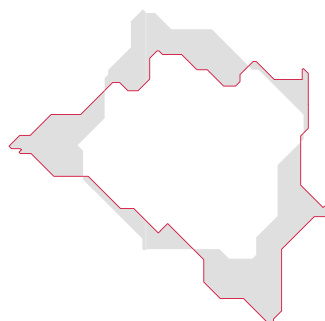
(d) $t = 0.33$



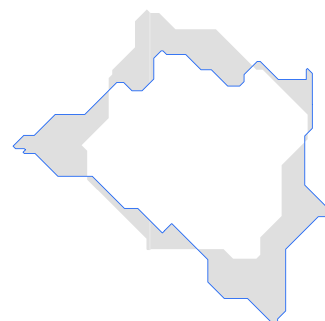
(e) $t = 0.66$



(f) $t = 0.66$

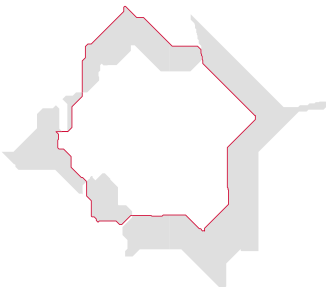


(g) $t = 1.00$

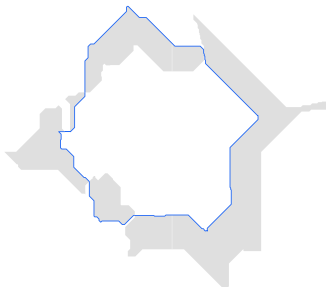


(h) $t = 1.00$

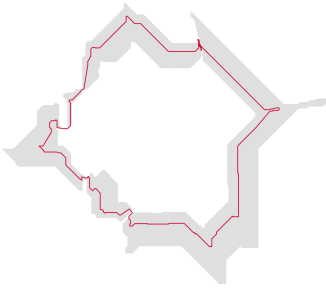
Fig. 19 Morph between two isochrones generated for different routing profiles. The morph consists of six subinstances. Baseline (left) and our approach (right)



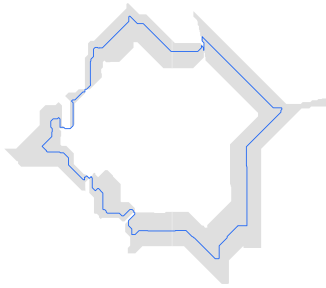
(a) $t = 0.00$



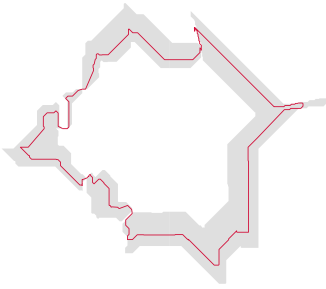
(b) $t = 0.00$



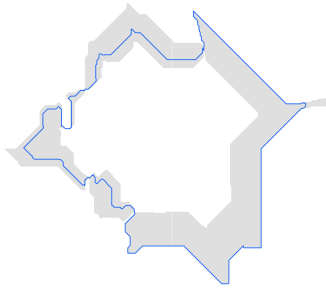
(c) $t = 0.33$



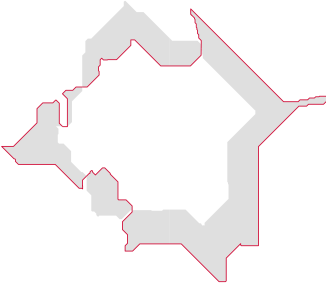
(d) $t = 0.33$



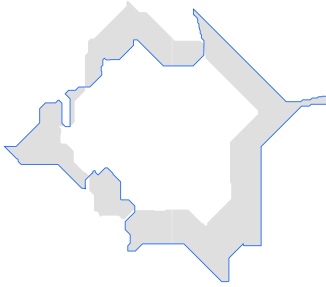
(e) $t = 0.66$



(f) $t = 0.66$



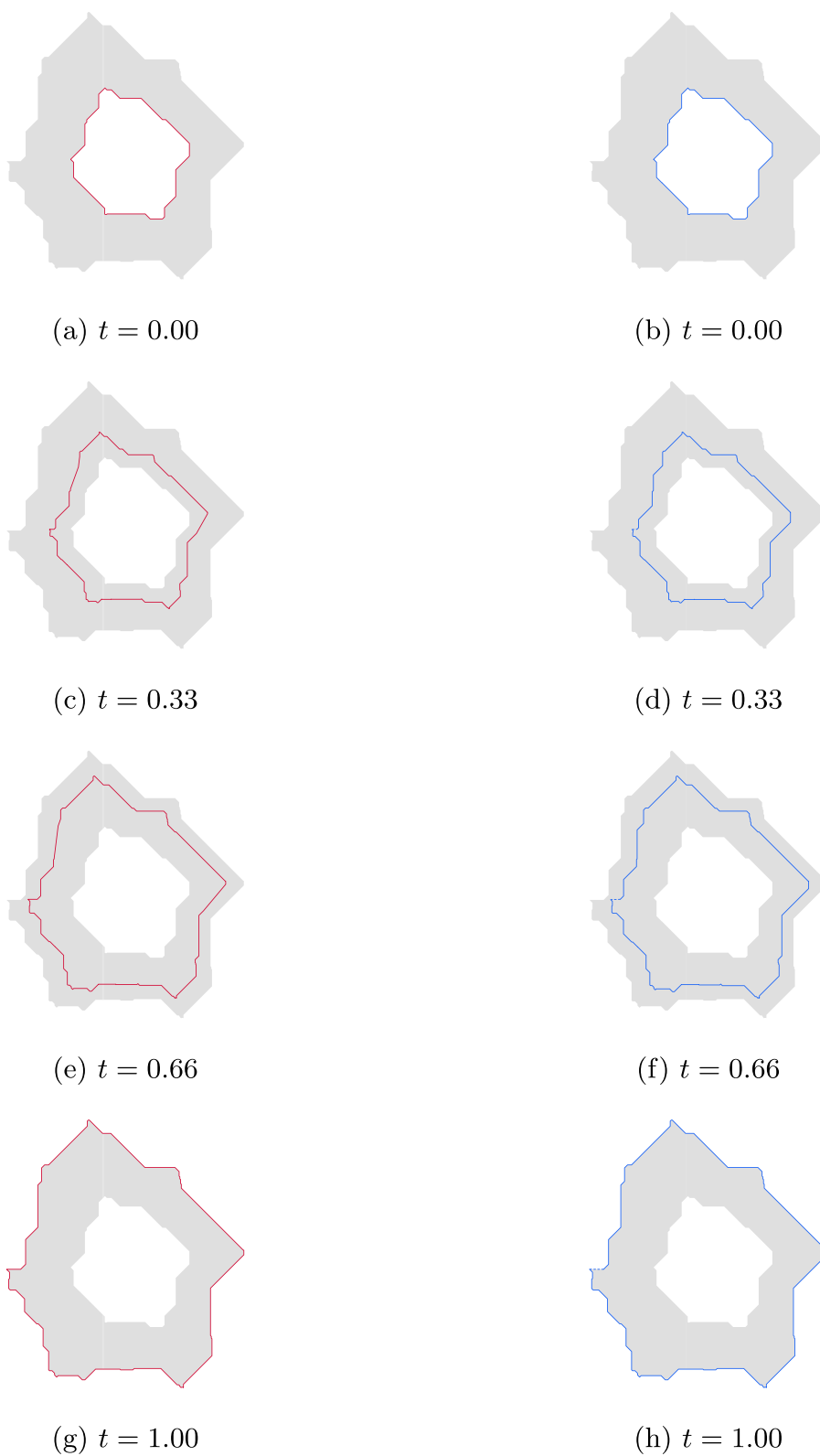
(g) $t = 1.00$



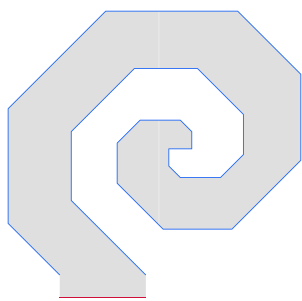
(h) $t = 1.00$

Appendix 2. Isochrones — Max Effort

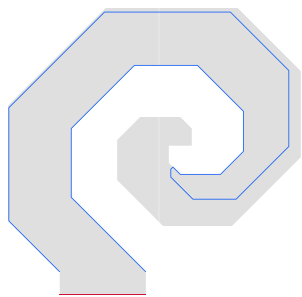
Fig. 20 Morph between two isochrones for different values of maximum effort. The morph is a single instance. Baseline (left) and our approach (right)



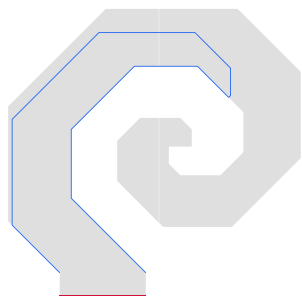
Appendix 3. Miscellaneous



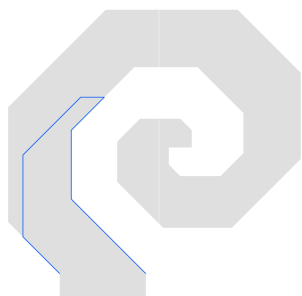
(a) $t = 0.00$



(b) $t = 0.025$

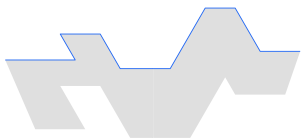


(c) $t = 0.10$



(d) $t = 0.30$

Fig. 21 Morphing a spiral to a straight line. The target ($t = 1.0$) is shown in red



(a) $t = 0.00$



(b) $t = 0.33$



(c) $t = 0.66$



(d) $t = 1.00$

Fig. 22 Hexalinear input polylines. The schematization is preserved

Appendix 4. Schematizing Polylines

This section describes the grid-based method we used to generate schematized polylines from a given input polyline. Specifically, this method generates the input for the river dataset used in the evaluation of our morphing method (“Evaluation” section).

Grid Graph We define an infinitely large octilinear grid graph from which we can generate a solution space for the polyline segments. The grid graph $\mathcal{G}_w = (\mathcal{V}, \mathcal{E})$ with grid spacing w is constructed as follows:

For each point $(i \cdot w, j \cdot w)$, where $i, j \in \mathbb{Z}$, we add a corresponding grid vertex $v_{i,j}$ to the vertex set $\mathcal{V}$. Every vertex $v_{i,j}$ is then connected to its eight octilinear neighbors by edges in the edge set $\mathcal{E}$.

This definition of a grid graph has already been used by Forsch and Haunert (2024). In this grid graph, any path $p = (e_1, \dots, e_n)$, with $e_i \in \mathcal{E}$ is an octilinear polyline. Thus, we can formulate the schematization problem as a routing problem in $\mathcal{G}$.

Edge Weighting Given an input polyline $\mathcal{P}$, we generate the schematized polyline by computing a minimum-cost path from the vertex $s \in \mathcal{G}$, which is the closest vertex to the starting point of $\mathcal{P}$ to the vertex $t \in \mathcal{G}$ closest to the end point of $\mathcal{P}$. To that end, we define a cost function $c : E \rightarrow \mathbb{R}$ to evaluate the suitability of each edge for the schematization. The cost is given by:

$$c(e) = \text{dist}(C(e), \mathcal{P}), \quad (10)$$

where $C(e)$ represents the midpoint of edge e , and dist returns the minimum Euclidean distance between $C(e)$ and polyline $\mathcal{P}$. This cost function ensures that edges closer to the original polyline $\mathcal{P}$ are preferred in the schematized path.

Additionally, the schematized polyline should not have many bends. By transforming the grid graph $\mathcal{G}$ into its pseudo-dual graph $\partial\mathcal{G}$, as introduced by Winter (2002), we can introduce a cost factor γ that penalizes bends.

In the octilinear setting, bends can occur at angles of 45° , 90° , or 135° . We assign a cost of $\gamma_{45} = w$, $\gamma_{90} = 2w$, and $\gamma_{135} = 3w$ for those bends, respectively. This means that a bend is introduced only if it reduces the cost c by at least one, two, or three times the grid width, depending on the bend angle.

Varying the Degree of Abstraction To generate multiple schematized versions of a polyline, each with a varying degree of abstraction, we can adjust the grid width w .

A smaller grid width results in a schematized polyline that closely follows the original input polyline $\mathcal{P}$, preserving

more of its detail. Conversely, increasing the grid width leads to a stronger schematization, where the polyline becomes more abstract and simplified, emphasizing the overall structure rather than the finer details.

By varying w , we can produce a range of schematized polylines, each reflecting a different balance between fidelity to the original shape and the level of abstraction.

Funding Open Access funding enabled and organized by Projekt DEAL. This research was funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation), grant no. 459420781.

Code and Data Availability The implementation, as well as the evaluation instances, are published as open source at <https://gitlab.igg.uni-bonn.de/rukemn/traveltime-maps>. Additionally, animated versions for all figures in this article can be found online at <https://www.geoinfo.uni-bonn.de/morphing>.

Declarations

On behalf of all authors, the corresponding author states that the manuscript was prepared in compliance with ethical standards.

Ethical Approval The manuscript does not report a study involving human participants.

Informed Consent The manuscript does not report a study involving human participants.

Conflict of Interest The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Alt H, Godau M (1995) Computing the Fréchet distance between two polygonal curves. *International Journal of Computational Geometry & Applications* 05(01n02):75–91. <https://doi.org/10.1142/S0218195995000064>
- Archambault D, Purchase HC (2013) Mental map preservation helps user orientation in dynamic graphs. In: *Proc. 20th international symposium on graph drawing and network visualization (GD'13)*, pp 475–486. https://doi.org/10.1007/978-3-642-36763-2_42
- Archambault D, Purchase HC (2016) Can animation support the visualisation of dynamic graphs? *Inf Sci* 330:495–509. <https://doi.org/10.1016/j.ins.2015.04.017>
- Bast H, Brosi P, Storandt S (2020) Metro maps on octilinear grid graphs. *Computer Graphics Forum* 39(3):357–367. <https://doi.org/10.1111/cgf.13986>

- Battersby SE, Goldsberry KP (2010) Considerations in design of transition behaviors for dynamic thematic maps. *Cartographic Perspectives* 65:16–32. <https://doi.org/10.14714/CP65.127>
- Bederson BB, Boltman A (1999) Does animation help users build mental maps of spatial information? In: *Proc. 1999 IEEE Symposium on Information Visualization (InfoVis'99)*, pp 28–35. <https://doi.org/10.1109/INFVIS.1999.801854>
- Bereg S (2005) An approximate morphing between polylines. *International Journal of Computational Geometry & Applications* 15(02):193–208. <https://doi.org/10.1142/S0218195905001658>
- Bespamyatnikh S (2002) An optimal morphing between polylines. *International Journal of Computational Geometry & Applications* 12(03):217–228. <https://doi.org/10.1142/S0218195902000839>
- Bespamyatnikh S (2003) An approximate morphing between polylines. In: *Proc. 1st International Conference on Computational Science and Its Applications (ICCSA'03)*, pp 807–816. https://doi.org/10.1007/3-540-44842-X_82
- Bonerath A, Niedermann B, Haunert JH (2019) Retrieving α -shapes and schematic polygonal approximations for sets of points within queried temporal ranges. In: *Proc. 27th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems (SIGSPATIAL'19)*, pp 249–258. <https://doi.org/10.1145/3347146.3359087>
- Brosi P, Bast H (2024) Large-scale generation of transit maps from OpenStreetMap data. *The Cartographic Journal* pp 1–25. <https://doi.org/10.1080/00087041.2024.2325761>
- Böttger J, Brandes U, Deussen O et al (2008) Map warping for the annotation of metro maps. *IEEE Comput Graphics Appl* 28(5):56–65. <https://doi.org/10.1109/MCG.2008.99>
- Buchin K, Meulemans W, Renssen AV et al (2016) Area-preserving simplification and schematization of polygonal subdivisions. *ACM Transactions on Spatial Algorithms and Systems* 2(1). <https://doi.org/10.1145/2818373>
- Buchin K, Buchin M, Meulemans W et al (2019) Locally correct Fréchet matchings. *Comput Geom* 76:1–18. <https://doi.org/10.1016/j.comgeo.2018.09.002>
- Casakin H (2004) Schematizing maps for wayfinding tasks: the role of 45° angular constraints, prototypical branching points and urban components. *J Spat Sci* 49(2):99–111. <https://doi.org/10.1080/14498596.2004.9635025>
- Cecconi A, Weibel R, Barrault M (2002) Improving automated generalisation for on-demand web mapping by multiscale databases. In: *Advances in Spatial Data Handling*, pp 515–531. https://doi.org/10.1007/978-3-642-56094-1_38
- Cicerone S, Cermignani M (2012) Fast and simple approach for polygon schematization. In: *Computational Science and Its Applications (ICCSA'12)*, pp 267–279. https://doi.org/10.1007/978-3-642-31125-3_21
- Delling D, Gemsa A, Nöllenburg M et al (2010) Path schematization for route sketches. In: *Algorithm Theory (SWAT'10)*, pp 285–296. https://doi.org/10.1007/978-3-642-13731-0_27
- Deng M, Peng D (2015) Morphing linear features based on their entire structures. *Trans GIS* 19(5):653–677. <https://doi.org/10.1111/TGIS.12111>
- Destler N, Singh M, Feldman J (2019) Shape discrimination along morph-spaces. *Vision Research* 158:189–199. <https://doi.org/10.1016/j.visres.2019.03.002>
- Dewulf B, Neutens T, Vanlommel M et al (2015) Examining commuting patterns using floating car data and circular statistics: exploring the use of new methods and visualizations to study travel times. *J Transp Geogr* 48:41–51. <https://doi.org/10.1016/j.jtrangeo.2015.08.006>
- Diehl S, Görg C, Kerren A (2001) Preserving the mental map using foresighted layout. *Data Visualization* 2001:175–184. https://doi.org/10.1007/978-3-7091-6215-6_19
- Dransch D (2001) Dynamic mapping in geography. In: *International encyclopedia of the social & behavioral sciences*. Pergamon, p 3908–3911. <https://doi.org/10.1016/B0-08-043076-7/02536-5>
- Efrat A, Har-Peled S, Guibas LJ et al (2001) Morphing between polylines. In: *Proc. 12th annual ACM-SIAM Symposium on Discrete Algorithms (SODA'01)*, vol 1. Society for Industrial and Applied Mathematics, pp 680–689. <https://doi.org/10.5555/365411.365564>
- Efrat A, Guibas LJ, Har-Peled S et al (2002) New similarity measures between polylines with applications to morphing and polygon sweeping. *Discrete & Computational Geometry* 28(4):535–569. <https://doi.org/10.1007/s00454-002-2886-1>
- Fink E, Wood D (2004) Restricted-orientation convexity. *Monographs in Theoretical Computer Science. An EATCS Series*, Springer. <https://doi.org/10.1007/978-3-642-18849-7>
- Forsch A, Oehrlin JH (2024) Metrochrones: schematic isochrones for schematic metro maps. *Cartogr J* 1–19. <https://doi.org/10.1080/00087041.2023.2284436>
- Forsch A, Dehbi Y, Niedermann B et al (2021) Multimodal travel-time maps with formally correct and schematic isochrones. *Trans GIS* 25(6):3233–3256. <https://doi.org/10.1111/tgis.12821>
- Forsch A, Oehrlin J, Niedermann B et al (2023) Inferring routing preferences from user-generated trajectories using a compression criterion. *Journal of Spatial Information Science* 26(5):99–124. <https://doi.org/10.5311/JOSIS.2023.26.256>
- Galvão ML, Krukar J, Schwering A (2021) Evaluating schematic route maps in wayfinding tasks for in-car navigation. *Cartogr Geogr Inf Sci* 48(5):449–469. <https://doi.org/10.1080/15230406.2021.1943531>
- Galvão ML, Krukar J, Schwering A (2023) Schematizing car routes with their surrounding street network. *Cartogr Geogr Inf Sci* 50(1):20–43. <https://doi.org/10.1080/15230406.2022.2125077>
- Gao A, Li J, Chen K (2020) A morphing approach for continuous generalization of linear map features. *PLoS ONE* 15(12):1–17. <https://doi.org/10.1371/journal.pone.0243328>
- Godfrey L, Mackaness W (2017) The bounds of distortion: truth, meaning and efficacy in digital geographic representation. *International Journal of Cartography* 3. <https://doi.org/10.1080/23729333.2017.1301348>
- Gortana F, Kaim S, von Lupin M et al (2014) Isoscope-visualizing temporal mobility variance with isochrone maps. *Poster Abstracts of IEEE VIS 2014*
- Gotsman C, Surazhsky V (2001) Guaranteed intersection-free polygon morphing. *Computers & Graphics* 25(1):67–75. [https://doi.org/10.1016/S0097-8493\(00\)00108-4](https://doi.org/10.1016/S0097-8493(00)00108-4)
- Guibas LJ, Hershberger J (1989) Optimal shortest path queries in a simple polygon. *J Comput Syst Sci* 39(2):126–152. [https://doi.org/10.1016/0022-0000\(89\)90041-X](https://doi.org/10.1016/0022-0000(89)90041-X)
- Harrower M (2001) Visualizing change: using cartographic animation to explore remotely-sensed data. *Cartographic Perspectives* 39:30–42. <https://doi.org/10.14714/CP39.637>
- Harrower M (2003) Tips for designing effective animated maps. *Cartographic Perspectives* 44:63–65. <https://doi.org/10.14714/CP44.516>
- Hartendorp MO, Van der Stigchel S, Burnett HG et al (2010) Categorical perception of morphed objects using a free-naming experiment. *Vis Cogn* 18(9):1320–1347. <https://doi.org/10.1080/13506285.2010.482774>
- Jenny B (2006) Geometric distortion of schematic network maps. *Bulletin of the Society of Cartographers* 40(1):15–18
- Kada M, Wichmann A, Hermes T (2015) Smooth transformations between generalized 3D building models for visualization purposes. *Cartogr Geogr Inf Sci* 42(4):306–314. <https://doi.org/10.1080/15230406.2015.1039588>
- Kekre HB, Sarode TK, Patil SM (2011) A novel pixel based color transition method for 2d image morphing. In: *Proc. Interna-*

- tional Conference & Workshop on Emerging Trends in Technology (ICWET'11). ACM, pp 357–362, <https://doi.org/10.1145/1980022.1980100>
- Kitchin RM (1994) Cognitive maps: what are they and why study them? *J Environ Psychol* 14(1):1–19. [https://doi.org/10.1016/S0272-4944\(05\)80194-X](https://doi.org/10.1016/S0272-4944(05)80194-X)
- Klippel A, Richter KF, Barkowsky T et al (2005) The cognitive reality of schematic maps. In: *Map-based mobile services: theories, methods and implementations*. Springer, p 55–71, https://doi.org/10.1007/3-540-26982-7_5
- Lan T, Li Z, Ti P (2019) Integrating general principles into mixed-integer programming to optimize schematic network maps. *Int J Geogr Inf Sci* 33(11):2305–2333. <https://doi.org/10.1080/13658816.2019.1620237>
- Li J, Mao K (2023) The morphing for continuous generalization of linear features based on dynamic time warping distance. *Geocarto Int* 38(1):2197516. <https://doi.org/10.1080/10106049.2023.2197516>
- Li J, Ai T, Liu P, et al (2017a) Continuous scale transformations of linear features using simulated annealing-based morphing. *ISPRS International Journal of Geo-Information* 6(8). <https://doi.org/10.3390/ijgi6080242>
- Li J, Li X, Xie T (2017b) Morphing of building footprints using a turning angle function. *ISPRS International Journal of Geo-Information* 6(6). <https://doi.org/10.3390/ijgi6060173>
- Li J, Liu P, Yu W et al (2018) The morphing of geographical features by Fourier transformation. *PLoS ONE* 13(1):1–13. <https://doi.org/10.1371/journal.pone.0191136>
- Li Z (2015) General principles for automated generation of schematic network maps. *Cartogr J* 52(4):356–360. <https://doi.org/10.1080/00087041.2015.1108661>
- Li Z, Dong W (2010) A stroke-based method for automated generation of schematic network maps. *Int J Geogr Inf Sci* 24(11):1631–1647. <https://doi.org/10.1080/13658811003766936>
- Lin H, Gong W (2018) Gradually morphing a thematic map series based on cellular automata. *Int J Geogr Inf Sci* 32(1):102–119. <https://doi.org/10.1080/13658816.2017.1379083>
- Liu L, Wang G, Zhang B et al (2004) Perceptually based approach for planar shape morphing. In: *Proc. 12th pacific conference on computer graphics and applications*. IEEE, pp 111–120, <https://doi.org/10.1109/PCCGA.2004.1348341>
- Lobo MJ, Pietriga E, Appert C (2015) An evaluation of interactive map comparison techniques. In: *Proc. 33rd annual ACM conference on human factors in computing systems (CHI'15)*. ACM, pp 3573–3582, <https://doi.org/10.1145/2702123.2702130>
- Meilinger T, Hölscher C, Büchner SJ et al (2007) How much information do you need? Schematic maps in wayfinding and self localisation. In: *Spatial cognition V: reasoning, action, interaction*. Springer, no. 4387 in *Lecture Notes in Artificial Intelligence*, pp 381–400
- Meulemans W (2014) Similarity measures and algorithms for cartographic schematization. PhD thesis, Technische Universiteit Eindhoven
- Meulemans W, van Renssen A, Speckmann B (2010) Area-preserving subdivision schematization. In: *Geographic information science*, pp 160–174, https://doi.org/10.1007/978-3-642-15300-6_12
- Michail D, Kinable J, Naveh B et al (2020) JGraphT—a java library for graph data structures and algorithms. *ACM Transactions on Mathematical Software* 46(2):1–19. <https://doi.org/10.1145/3381449>
- Morrison JB (2000) Does animation facilitate learning? An evaluation of the congruence and equivalence hypotheses. PhD thesis, Stanford University
- Nöllenburg M, Terziadis S (2024) Computing data-driven multilinear metro maps. *Cartogr J* 1–16. <https://doi.org/10.1080/00087041.2024.2304476>
- Nöllenburg M, Wolff A (2011) Drawing and labeling high-quality metro maps by mixed-integer programming. *IEEE Trans Visual Comput Graphics* 17(5):626–641. <https://doi.org/10.1109/TVCG.2010.81>
- Nöllenburg M, Merrick D, Wolff A, et al (2008) Morphing polylines: a step towards continuous generalization. *Computers, Environment and Urban Systems* 32(4):248–260. <https://doi.org/10.1016/j.compenvurbsys.2008.06.004>, geographical Information Science Research – United Kingdom
- Panis S, Vangeneugden J, Wagemans J (2008) Similarity, typicality, and category-level matching of morphed outlines of everyday objects. *Perception* 37(12):1822–1849. <https://doi.org/10.1068/p5934>
- Pantazis DN, Karathanasis B, Kassoli M et al (2009) Are the morphing techniques useful for cartographic generalization? In: *Urban and regional data management*. CRC Press, pp 207–216, <https://doi.org/10.1201/9780203869352-22>
- Peng D, Meijers M, van Oosterom P (2023) Generalizing simultaneously to support smooth zooming: case study of merging area objects. *Journal of Geovisualization and Spatial Analysis* 7(1):12. <https://doi.org/10.1007/s41651-022-00109-x>
- Polisciuc E, Alves A, Bento C, et al (2013) Visualizing urban mobility. In: *ACM SIGGRAPH 2013 Posters (SIGGRAPH'13)*, p 1, <https://doi.org/10.1145/2503385.2503511>
- Rahman MT, Al-Amin M, Bakkre JB, et al (2007) A novel approach of image morphing based on pixel transformation. In: *2007 10th international conference on computer and information technology*. IEEE, pp 1–5, <https://doi.org/10.1109/ICCITECHN.2007.4579398>
- Roberts MJ (2023) Objective and subjective methods for evaluating the usability of schematic maps: the case against informal expert assessments. *Cartogr J* 1–18. <https://doi.org/10.1080/00087041.2023.2246742>
- Sederberg TW, Greenwood E (1992) A physically based approach to 2–D shape blending. In: *Proc. 19th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH'92)*. ACM, pp 25–34, <https://doi.org/10.1145/133994.134001>
- Sedig K, Rowhani S, Liang HN (2005) Designing interfaces that support formation of cognitive maps of transitional processes: an empirical study. *Interact Comput* 17(4):419–452. <https://doi.org/10.1016/j.intcom.2005.02.002>
- Sester M, Brenner C (2005) Continuous generalization for visualization on small mobile devices. In: *Developments in Spatial Data Handling*. Springer, pp 355–368, https://doi.org/10.1007/3-540-26772-7_27
- Speckmann B, Verbeek K (2018) Homotopic c-oriented routing with few links and thick edges. *Comput Geom* 67:11–28. <https://doi.org/10.1016/j.comgeo.2017.10.005>
- Surazhsky V, Gotsman C (2001) Controllable morphing of compatible planar triangulations. *ACM Trans Graph* 20(4):203–231. <https://doi.org/10.1145/502783.502784>
- Surazhsky V, Gotsman C (2001b) Morphing stick figures using optimized compatible triangulations. In: *Proc. 9th pacific conference on computer graphics and applications*. IEEE, Pacific Graphics 2001, pp 40–49, <https://doi.org/10.1109/PCCGA.2001.962856>
- Tang KT (2007) Vector calculus. In: *Mathematical methods for engineers and scientists 2: vector analysis, ordinary differential equations and Laplace transforms*. Springer, pp 35–112, https://doi.org/10.1007/978-3-540-30270-4_2
- Ti P, Wu H, Li Z et al (2023) Revealing schematic map designs with preservation of relativity in node position and segment length in existing official maps. *ISPRS International Journal of Geo-Information* 12(8). <https://doi.org/10.3390/ijgi12080309>
- Turton I (2008) Geo tools. In: *Open source approaches in spatial data handling*. Advances in Geographic Information Science, vol 2 (AGIS), Springer, chap 8, pp 153–169, https://doi.org/10.1007/978-3-540-74831-1_8
- Tversky B, Morrison JB, Betrancourt M (2002) Animation: can it facilitate? *Int J Hum Comput Stud* 57(4):247–262. <https://doi.org/10.1006/ijhc.2002.1017>

- van Dijk TC, Lutz D (2018) Realtime linear cartograms and metro maps. In: Proc. 26th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems (SIGSPATIAL'18), pp 488–491, <https://doi.org/10.1145/3274895.3274959>
- van Kreveld M (2001) Smooth generalization for continuous zooming. In: Proc. 20th International Geographic Conference (ICC'01). ICA, pp 2180–2185
- van Kreveld M, Miltzow T, Ophelders T et al (2022) Between shapes, using the Hausdorff distance. *Comput Geom* 100. <https://doi.org/10.1016/j.comgeo.2021.101817>
- van Oosterom P, Meijers M (2014) Vario-scale data structures supporting smooth zoom and progressive transfer of 2D and 3D data. *Int J Geogr Inf Sci* 28(3):455–478. <https://doi.org/10.1080/13658816.2013.809724>
- Visser H, de Nijs T (2006) The map comparison kit. *Environmental Modelling & Software* 21(3):346–358. <https://doi.org/10.1016/j.envsoft.2004.11.013>
- Wang YS, Chi MT (2011) Focus+context metro maps. *IEEE Trans Visual Comput Graphics* 17(12):2528–2535. <https://doi.org/10.1109/TVCG.2011.205>
- Winter S (2002) Modeling costs of turns in route planning. *GeoInformatica* 6(4):345–361. <https://doi.org/10.1023/A:1020853410145>
- Wu HY, Niedermann B, Takahashi S et al (2020) A survey on transit map layout - from design, machine, and human perspectives. *Computer Graphics Forum* 39(3):619–646. <https://doi.org/10.1111/cgf.14030>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.